HOTEL HAVEN

Hotel Room Reservation System

C++ Object-Oriented Programming Mini Project

PROJECT REPORT

Submitted by: Swayam Krupesh Nakhwa

Roll No.: BT24ECE080

Course: Introduction to Object Oriented Programming
Semester: 5th Semester

Guided by: Aparna Wamanrao Bondade Ma'am

Indian Institute of Information Technology Nagpur

Academic Year: 2026

Abstract

Hotel Haven is a locally hosted hotel room reservation web application developed as an Object-Oriented Programming mini project. It combines a C++ server and reservation domain with a responsive HTML, CSS, and JavaScript interface. A visitor can browse six sample rooms, check availability for a date range, create a reservation, look up a reservation by email, and cancel it using its reference and email. Booking records are saved to a CSV file so they can be loaded again when the server restarts.

The project demonstrates encapsulation, abstraction, inheritance, runtime polymorphism, composition, separation of concerns, date validation, overlap detection, and file persistence. It is intended for learning and demonstration on a local Windows machine. It does not provide secure user accounts, payment processing, or production-grade hosting.

Contents

- 1. Introduction and problem statement
- 2. Objectives, scope, and requirements
- 3. System design and architecture
- 4. Object-oriented design
- 5. Reservation workflow and business rules
- 6. User interface and accessibility
- 7. Implementation and project structure
- 8. Testing and results
- 9. Installation and user guide
- 10. Limitations and future enhancements
- 11. Conclusion and references

1. Introduction and problem statement

Hotel room booking involves selecting a room, entering stay dates and guest information, checking availability, calculating a price, and managing an existing reservation. This project models that workflow in a small educational application. The main challenge is to prevent invalid or overlapping reservations while keeping the C++ domain logic understandable and separated from HTTP and presentation code.

1.1 Proposed solution

The application provides a browser interface served by a C++ HTTP server on the local machine. The frontend sends requests to the server; a BookingService validates input and coordinates room and booking objects with a repository. The CSV repository persists reservations between runs.

2. Objectives, scope, and requirements

- Demonstrate core C++ OOP concepts through a coherent application.
- Display six rooms across Standard, Deluxe, and Suite categories.
- Search date-based availability, create a reservation, look it up, and cancel it.
- Validate dates, guest capacity, email format, and room conflicts.
- Persist reservation records in CSV and reload them after restart.
- Provide a responsive, keyboard-accessible interface with light and dark themes.

2.1 Scope and assumptions

The application is a local classroom demonstration. Prices are illustrative and shown in INR; no payment is taken. The system uses a predefined sample room catalog and one local CSV data file. It does not implement accounts, role-based access, online deployment, a relational database, or concurrent production traffic.

2.2 Functional requirements

ID	Requirement	Implemented behavior
FR-01	Browse rooms	Show six sample rooms, category, capacity, nightly rate, amenities, and photos.
FR-02	Search availability	Return rooms that have no confirmed overlapping reservation for the requested stay.
FR-03	Create reservation	Validate inputs, calculate total, assign a reference, and persist the booking.
FR-04	Manage reservation	Find reservations by email and cancel using reference plus matching email.
FR-05	Retain records	Load and save booking records through a CSV repository.

2.3 Non-functional requirements

Area	Design choice / status
Usability	Responsive layouts, larger text, room imagery, theme switch, and clear feedback.
Accessibility	Labels, alternative text, keyboard focus, skip link, reduced-motion handling, and slideshow controls.
Maintainability	Separate domain, service, repository, HTTP, frontend, and tests.
Performance	Small local demonstration with cached assets and local photos; server handles one request at a time.
Portability	Current server and batch scripts target Windows and Winsock.

3. System design and architecture

The application is organized in layers. Browser presentation is separated from the C++ domain and service rules. HTTP request handling translates browser requests into service operations, while the repository abstraction keeps persistence behind a small interface.

Layer	Main components	Responsibility
Presentation	public/index.html, styles.css, app.js	Room display, forms, themes, slideshow, and interaction feedback.
Transport	HttpServer	Serve static assets and JSON API responses over local HTTP.
Application	BookingService	Coordinate validation, room availability, creation, lookup, and cancellation.
Domain	Room, Booking	Represent room categories, guest stays, pricing, references, and booking state.
Persistence	BookingRepository, CsvBookingRepository	Abstract loading/saving and implement CSV storage.
Verification	tests/booking_service_tests.cpp	Exercise reservation rules and CSV round-trip behavior.

3.1 Request flow

Browser UI -> local HTTP endpoint -> BookingService -> Room / Booking domain objects -> BookingRepository -> data/bookings.csv. The response returns to the browser as JSON, where the interface displays the result.

4. Object-oriented design

OOP concept	Project example	Why it is useful
Encapsulation	Room and Booking store private fields and expose intentional getters/operations.	Protects object state and centralizes rules.
Abstraction	Room has a virtual category contract; BookingRepository defines load/save behavior.	Callers use contracts without depending on implementation details.
Inheritance	StandardRoom, DeluxeRoom, and SuiteRoom derive from Room.	Shares common room information while representing distinct categories.
Runtime polymorphism	Room catalog stores shared_ptr<Room>; category() dispatches to the concrete room type.	Enables uniform handling of different room categories.
Composition / collaboration	BookingService uses room objects and a repository reference.	Separates responsibilities while coordinating a complete workflow.
Separation of concerns	HTTP, business rules, CSV persistence, and browser code live separately.	Makes changes easier to locate and test.

4.1 Key classes

- Room: common room identity, capacity, rate, description, amenities, and virtual category.
- Booking: reservation reference, guest and stay details, total price, and booking status.
- BookingService: business operations and input/availability validation.
- BookingRepository: persistence contract; CsvBookingRepository handles CSV serialization.
- HttpServer: local web server, static assets, and JSON API connection.

5. Reservation workflow and business rules

1. The visitor selects check-in and check-out dates and searches for available rooms.
2. The server validates dates and searches the room catalog against active reservations.
3. The visitor selects a room, guest count, guest name, and email to submit a booking.
4. BookingService checks required values, email format, date order, guest capacity, room identity, and date overlap.
5. For a valid stay, the total is nightly rate multiplied by the number of nights; a reference is assigned and the booking is saved.
6. For management, the visitor supplies an email to find reservations; cancellation requires the booking reference and matching email.

5.1 Availability rule

A reservation occupies a room when the requested stay overlaps its active stay. Check-out and the next check-in on the same date are treated as adjacent rather than overlapping. Cancelled reservations do not block availability.

6. User interface and accessibility

The interface presents room photography, a hero slideshow for Classic King, Deluxe Garden, and The Corner Suite, room cards, search and booking forms, and reservation management. CSS adapts the layout for mobile, tablet, and desktop widths. Theme preference is retained in the browser.

- Form labels and image alternative text support assistive technologies.
- Keyboard focus styles and a skip link support keyboard navigation.
- Slideshow has manual controls, pause/resume, and reduced-motion support.
- Color theme and larger text improve visual flexibility, but do not guarantee accessibility for every user.

Desktop interface

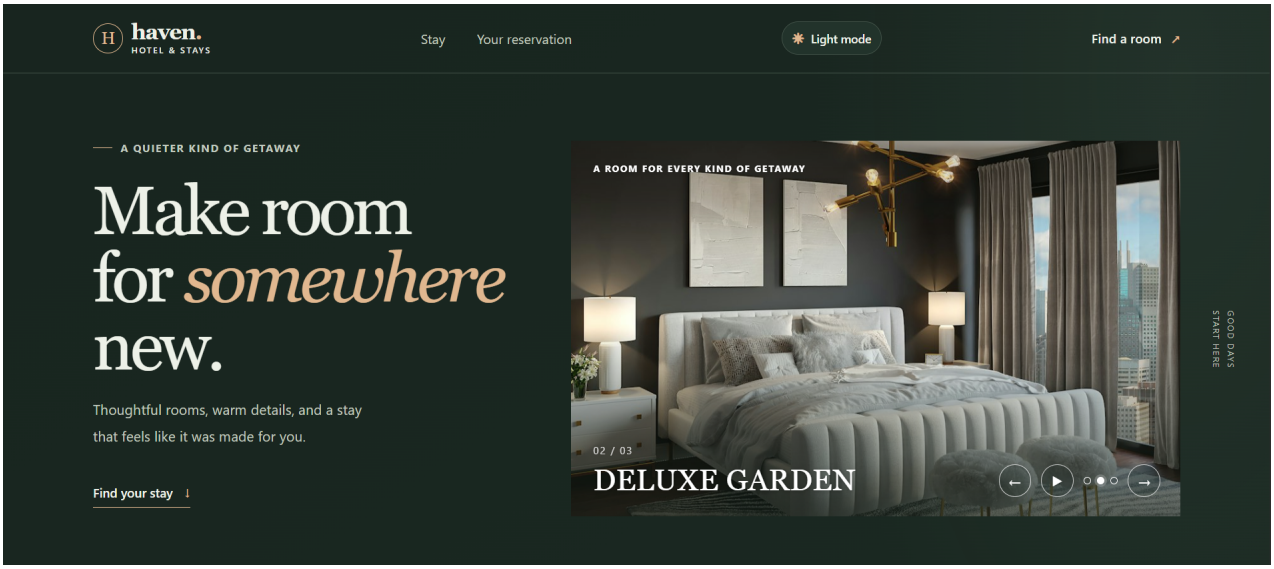


Figure 1. Hotel Haven desktop header and room slideshow.

Mobile interface

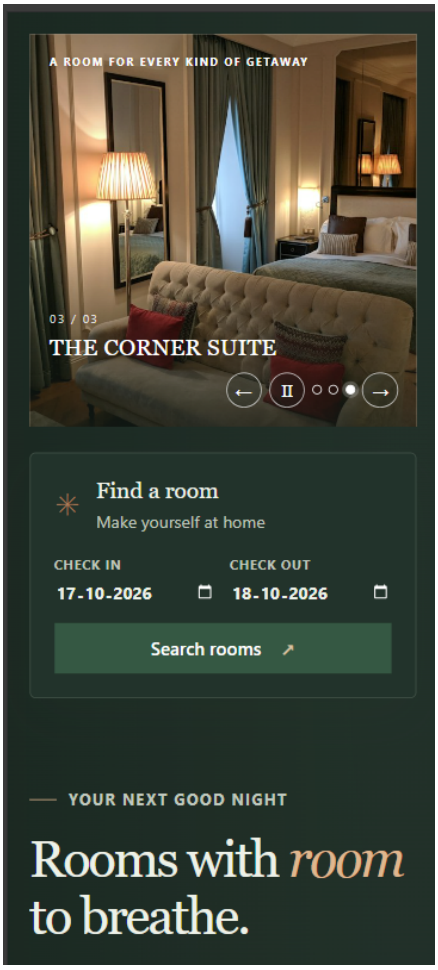


Figure 2. Mobile layout showing slideshow and room search.

7. Implementation and project structure

Path	Purpose
include/	C++ declarations for domain, service, repository, and server.
src/	C++ implementations and application entry point.
public/	HTML, CSS, JavaScript, and locally served room photographs.
tests/	Reservation service test program.
build.bat / run.bat	Build the C++ server and start it on Windows.
test.bat	Compile and run the C++ tests.
data/bookings.csv	Runtime reservation data; excluded from Git to avoid sharing personal/test data.

The project builds with a C++14-compatible MinGW g++ compiler and links Winsock using -lws2_32. No JavaScript package manager or external C++ library is required.

8. Testing and results

The included dependency-free C++ test executable covers date handling, room availability, reservation validation, cancellation, and CSV persistence. The test program reports success only after all assertions pass.

Test area	Representative check	Expected result
Dates	Leap-day validity, invalid month/day, night calculation.	Valid dates accepted; invalid dates rejected; nights counted correctly.
Booking creation	Reference, total, persistence, normalized email lookup.	Correct reference and total; record saved and found.
Availability	Overlapping stay and same-day check-out/check-in boundary.	Overlapping stay unavailable; adjacent stay allowed.
Validation	Invalid dates, malformed email, guest count above capacity.	Invalid request rejected.
Cancellation	Wrong email, successful cancellation, repeated cancellation.	Wrong owner rejected; valid cancellation saved and room released.
CSV	Round trip with comma in guest name and cancellation status.	Data is correctly serialized and reloaded.

Run the tests from the project folder with .\test.bat. Run .\build.bat to compile the server. These checks validate business behavior; they are not a production load or penetration test.

9. Installation and user guide

1. Install Git and a MinGW C++ compiler; ensure g++ is available in PATH.
2. Clone the repository: git clone https://github.com/Swayam-nakhwa/hotel-haven-app.git
3. Open a terminal in the cloned hotel-haven-app folder.
4. Start the server with .\run.bat. This builds and starts the C++ application.
5. Visit http://127.0.0.1:8080 in a browser. Keep the terminal open while using the app; press Ctrl+C to stop.
6. Run .\test.bat to execute the reservation tests.

If port 8080 is occupied, run .\run.bat 8090 and use http://127.0.0.1:8090. The server is intended for local Windows use and is not a public service.

10. Limitations and future enhancements

Current limitation	Possible future enhancement
No authentication; email plus reference is demo-level lookup.	Add secure account identity, authorization, and password handling if expanding the scope.
CSV storage is local and unsuitable for concurrent production use.	Move to a transactional database and add safe concurrent access.
Single-threaded small demonstration HTTP server.	Use a production-grade server framework and load testing before public hosting.
Windows Winsock and batch build scripts.	Add a portable networking layer and cross-platform build configuration.
No payment gateway, email notifications, or admin console.	Add these only with proper security, privacy, and operational design.
SEO ranking cannot be evaluated on localhost.	Deploy to a domain and add canonical/social preview metadata before assessing indexing.

11. Conclusion and references

Hotel Haven turns core OOP concepts into a complete, understandable reservation workflow. It connects a responsive browser UI to a C++ service and domain model, validates booking rules, and persists records through a CSV repository. The project is suitable as an educational portfolio demonstration when its local-only and non-production limitations are clearly stated.

References

- Project source and instructions: https://github.com/Swayam-nakhwa/hotel-haven-app
- C++ language standard used by the build script: C++14.
- Room imagery is included as local JPEG assets; source image URLs are documented in README.md.

Before submission, replace the bracketed identity fields on the cover page and confirm any institution-specific report format requirements.